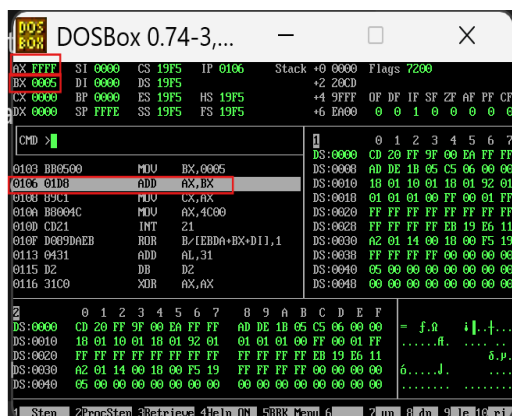


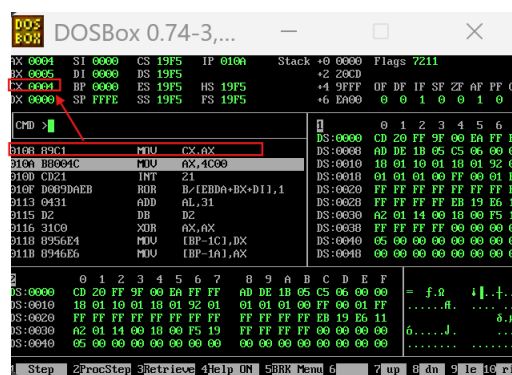
QUESTION #1 : MEMORY WRAP AROUND EXAMPLE:

```
1  org 0x0100
2
3  mov ax,0FFFFh;
4  mov bx, 0005h;
5  add ax,bx;
6  mov cx,ax;
7
8
9  mov ax,0x4c00;
10 int 0x21;
```



Physical memory wraparound:

As seen above both AX and BX store 16-bit value but when we add them the value(0x10004) gets out of bound causing and address wraparound thus when we add ax and bx , the result in cx is unexpected .



as explained earlier result can be seen here.

TASK #2: 5 conditional jumps (any):

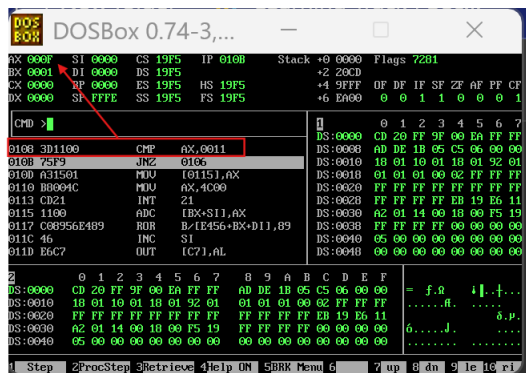
1. JNE:

```
org 0x0100

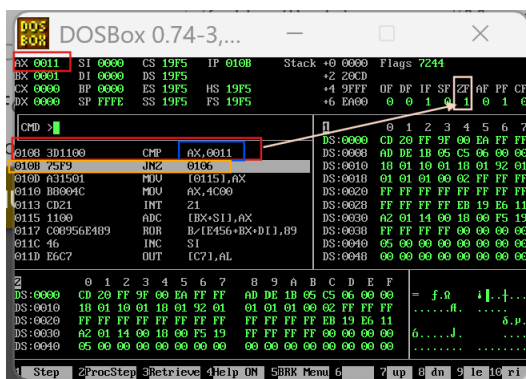
mov ax, 14
mov bx, 1
start_loop:
    add ax, bx
    cmp ax, 15
    jne start_loop

mov [result], ax
mov ax, 0x4c00
int 0x21

result: dw 0
```



Over here compiler compares the value stored in AX register with the hex value of 17(11) when the will be equal ZF will be raised and the loop will stop else the loop continues:



Now in the red box we can see value of AX has been compared and has been matched this zero flag has been raised, in orange box zero flag is checked whether it's been raised or not to make a decision of continuing loop or not.

2. JE:

In the following code value in AX is compared with F if they will be equal zero flag will be raised in next step and jump will occur

DOSBox 0.74-3,...

AX:0001 SI:0000 CS:19F5 IP:0100 Stack:0000 Flags:7204
BX:0001 DI:0000 DS:19F5 +2 20CD
CX:0000 BP:0000 ES:19F5 HS:19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX:0000 SP:FFFF SS:19F5 FS:19F5 +6 E000 0 0 1 0 0 0 1 0

CMD >

0106 01D8 ADD AX,BX DS:0000 CD 20 FF 9F 00 EA FF FF
0108 3D0F00 CMP AX,000F DS:0000 AD DE 1B 05 C5 06 00 00
0109 74F9 JZ 0106 DS:0010 1B 01 10 01 1B 01 92 01
010B 031501 MOV [0115],AX DS:0020 FF FF FF FF FF FF FF
0110 B9004C MOV AX,4C00 DS:0023 FF FF FF FF EB 19 E5 11
0113 CD21 INT 21 DS:0030 A2 01 14 00 1B 00 F5 19
0115 1000 ADC [BX+SI],AL DS:003B FF FF FF FF 00 00 00 00
0117 C08956E4B9 ROR B/[E456+BX+DI],09 DS:0040 05 00 00 00 00 00 00 00
011C 46 INC SI DS:004B 00 00 00 00 00 00 00 00

Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

In the previous step zero flag has been raised as shown below and now the program will jump back to the loop as ax is equal to cmp value

DOSBox 0.74-3,...

AX:000F SI:0000 CS:19F5 IP:010B Stack:0000 Flags:7244
BX:0001 DI:0000 DS:19F5 +2 20CD
CX:0000 BP:0000 ES:19F5 HS:19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX:0000 SP:FFFF SS:19F5 FS:19F5 +6 E000 0 0 1 1 0 0 1 0

CMD >

0106 3D0F00 CMP AX,000F DS:0000 CD 20 FF 9F 00 EA FF FF
0108 74F9 JZ 0106 DS:0010 1B 01 10 01 1B 01 92 01
010B 031501 MOV [0115],AX DS:0020 FF FF FF FF FF FF FF
0110 B9004C MOV AX,4C00 DS:0023 FF FF FF FF EB 19 E5 11
0113 CD21 INT 21 DS:0030 A2 01 14 00 1B 00 F5 19
0115 1000 ADC [BX+SI],AL DS:003B FF FF FF FF 00 00 00 00
0117 C08956E4B9 ROR B/[E456+BX+DI],09 DS:0040 05 00 00 00 00 00 00 00
011C 46 INC SI DS:004B 00 00 00 00 00 00 00 00
011D E0C7 OUT [C7],AL DS:004B 00 00 00 00 00 00 00 00

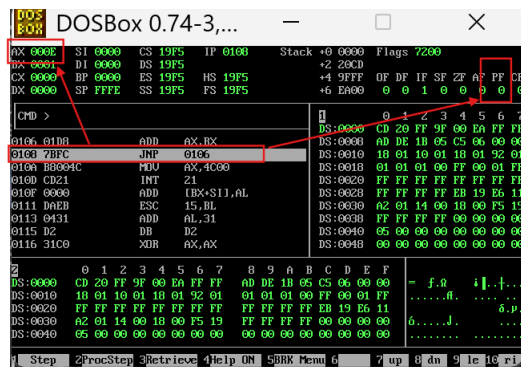
Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

3. JNP:

It would jump until the parity is even

```
1      org 0x0100
2
3      mov ax, 13
4      mov bx, 1
5      start_loop:
6          add ax, bx
7          jnp start_loop
8
9      mov ax, 0x4c00
10     int 0x21
11
12     result: dw 0
```

As shown below the loops checks whether the parity flag is raised or not in the resulting value. If the parity flag is 0 (As 13 results in odd parity) the loop will jump until it becomes even parity.



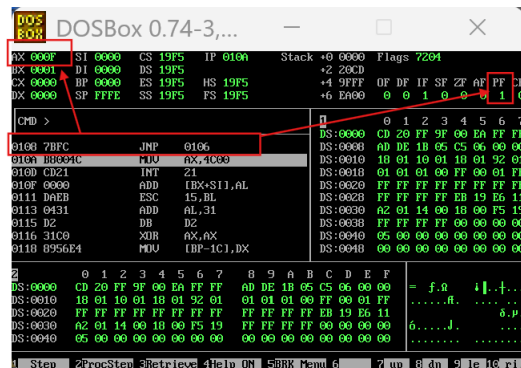
DOSBox 0.74-3,...

AX: 000F SI: 0000 CS: 19F5 IP: 0100 Stack: +0 0000 Flags: 7200
BX: 0001 DI: 0000 DS: 19F5 +2 20CD
CX: 0000 BP: 0000 ES: 19F5 HS: 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX: 0000 SP: FFFE SS: 19F5 FS: 19F5 +6 E000 0 0 1 0 0 0 0 0

0106 7BFC JNP 0106
010A B804C MOV AX, 4C00
010D CD21 INT 21
010F 0000 ADD [BX+SI], AL
0111 DAE8 ESC 15, BL
0113 0431 ADD AL, 31
0115 D2 DB D2
0116 31C0 XOR AX, AX

DS:0000 CD 20 FF 9F 00 EA FF FF = f..r i..+...
DS:0010 10 01 10 01 10 01 92 01 01 01 01 00 FF 00 01 FFf.
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 E6 11s.p.
DS:0030 A2 01 14 00 10 00 F5 19 FF FF FF FF 00 00 00 00 6.....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Now as PF has been raised after the AX register value is 15 (which is now even parity). So the jump won't take place now.



DOSBox 0.74-3,...

AX: 000F SI: 0000 CS: 19F5 IP: 0100 Stack: +0 0000 Flags: 7204
BX: 0001 DI: 0000 DS: 19F5 +2 20CD
CX: 0000 BP: 0000 ES: 19F5 HS: 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX: 0000 SP: FFFE SS: 19F5 FS: 19F5 +6 E000 0 0 1 0 0 0 1 0

0106 7BFC JNP 0106
010A B804C MOV AX, 4C00
010D CD21 INT 21
010F 0000 ADD [BX+SI], AL
0111 DAE8 ESC 15, BL
0113 0431 ADD AL, 31
0115 D2 DB D2
0116 31C0 XOR AX, AX
0118 856E4 MOV [BP+1C], DX

DS:0000 CD 20 FF 9F 00 EA FF FF = f..r i..+...
DS:0010 10 01 10 01 10 01 92 01 01 01 01 00 FF 00 01 FFf.
DS:0020 FF FF FF FF FF FF FF FF FF FF FF FF EB 19 E6 11s.p.
DS:0030 A2 01 14 00 10 00 F5 19 FF FF FF FF 00 00 00 00 6.....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

4. JP:

The following program checks the parity of ax register which currently stores value 9:

1st iteration: AX stores 9 the program jumps as PF flag is 1 and 9 has even parity.

```
DOSBox 0.74-3, ...
AX:0009 SI:0000 CS:19F5 IP:0103 Stack:0000 Flags:7204
BX:0001 DI:0000 DS:19F5
CX:0000 BP:0000 ES:19F5 HS:19F5
DX:0000 SP:FFFF SS:19F5 FS:19F5

[CMD >]
0103 7AFC JP 0106
010A B804C MOV AX,4C00
010F CD21 INT 21
010F 0000 ADD [BX:SI],AL
0111 D0EB ESC 15,BL
0113 0431 ADD AL,31
0115 D2 DB D2
0116 31C9 XOR AX,AX

[Registers]
AX:0009 CD 20 FF 9F 00 EA FF FF
BX:0001 AD DE 1B 05 C5 06 00 00
CX:0000 18 01 10 01 18 01 92 01
DX:0000 01 01 01 00 FF 00 01 00
SI:0000 01 FF FF FF FF FF FF FF
DI:0000 FF FF FF FF FF FF FF FF
BP:0000 01 FF FF FF FF FF FF FF
SP:FFFF 02 01 14 00 18 00 F5 19
FS:19F5 FF FF FF FF 00 00 00 00
ES:19F5 05 00 00 00 00 00 00 00

[Flags]
Z 0 A 1 2 3 4 5 6 7 8 9 A B C D E F
O 0 S 0 O 0 0 0 0 0 0 0 0 0 0 0 0
D 0 I 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
S 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
P 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
C 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
E 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
F 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

2nd iteration: AX now stores 10 parity is again even and PF is 1.

```
DOSBox 0.74-3, ...
AX:000A SI:0000 CS:19F5 IP:0106 Stack:0000 Flags:7204
BX:0001 DI:0000 DS:19F5
CX:0000 BP:0000 ES:19F5 HS:19F5
DX:0000 SP:FFFF SS:19F5 FS:19F5

[CMD >]
0103 7AFC JP 0106
010A B804C MOV AX,4C00
010F CD21 INT 21
010F 0000 ADD [BX:SI],AL
0111 D0EB ESC 15,BL
0113 0431 ADD AL,31
0115 D2 DB D2
0116 31C9 XOR AX,AX

[Registers]
AX:000A CD 20 FF 9F 00 EA FF FF
BX:0001 AD DE 1B 05 C5 06 00 00
CX:0000 18 01 10 01 18 01 92 01
DX:0000 01 01 01 00 FF 00 01 00
SI:0000 01 FF FF FF FF FF FF FF
DI:0000 FF FF FF FF FF FF FF FF
BP:0000 01 FF FF FF FF FF FF FF
SP:FFFF 02 01 14 00 18 00 F5 19
FS:19F5 FF FF FF FF 00 00 00 00
ES:19F5 05 00 00 00 00 00 00 00

[Flags]
Z 0 A 1 2 3 4 5 6 7 8 9 A B C D E F
O 0 S 0 O 0 0 0 0 0 0 0 0 0 0 0 0
D 0 I 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
S 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
P 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
C 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
E 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
F 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

3rd iteration: AX now stores 11 parity becomes odd so PF flag is zero and no more jump occurs.

```
DOSBox 0.74-3, ...
AX:000B SI:0000 CS:19F5 IP:010A Stack:0000 Flags:7200
BX:0001 DI:0000 DS:19F5
CX:0000 BP:0000 ES:19F5 HS:19F5
DX:0000 SP:FFFF SS:19F5 FS:19F5

[CMD >]
0103 7AFC JP 0106
010A B804C MOV AX,4C00
010F CD21 INT 21
010F 0000 ADD [BX:SI],AL
0111 D0EB ESC 15,BL
0113 0431 ADD AL,31
0115 D2 DB D2
0116 31C9 XOR AX,AX
011B 8956E4 MOV [BX-1C],DX

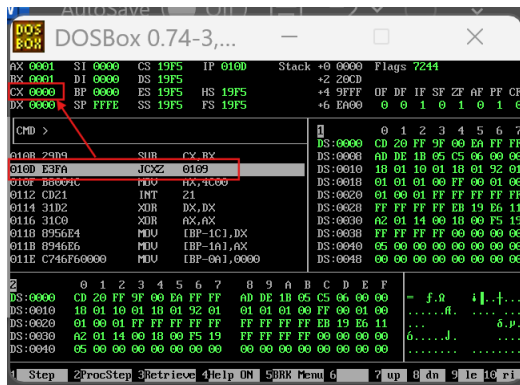
[Registers]
AX:000B CD 20 FF 9F 00 EA FF FF
BX:0001 AD DE 1B 05 C5 06 00 00
CX:0000 18 01 10 01 18 01 92 01
DX:0000 01 01 01 00 FF 00 01 00
SI:0000 01 FF FF FF FF FF FF FF
DI:0000 FF FF FF FF FF FF FF FF
BP:0000 01 FF FF FF FF FF FF FF
SP:FFFF 02 01 14 00 18 00 F5 19
FS:19F5 FF FF FF FF 00 00 00 00
ES:19F5 05 00 00 00 00 00 00 00

[Flags]
Z 0 A 1 2 3 4 5 6 7 8 9 A B C D E F
O 0 S 0 O 0 0 0 0 0 0 0 0 0 0 0 0
D 0 I 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
S 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
P 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
A 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
C 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
E 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
F 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

As you see the program went on next step and jump has not occurred this time.

5. JCXZ:

1st iteration: as cx stores 0 jump will occur as shown below :



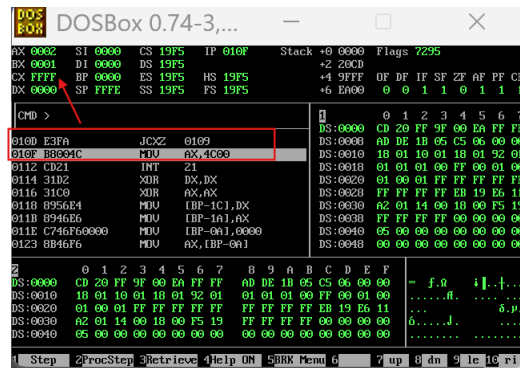
DOSBox 0.74-3,...

AX:0001 SI:0000 CS:19F5 IP:0100 Stack:+0 0000 Flags:7244
BX:0001 DI:0000 DS:19F5
CX:0000 BP:0000 ES:19F5 HS:19F5
DX:0000 SP:FFFF SS:19F5 FS:19F5

0100 2909 SUB CX,BX
010D E3FA JCXZ 0109
010F B0004C MOV AX,4C00
0112 CD21 INT ZI
0114 31D2 XDR DX,BX
0116 31C9 XDR AX,AX
011B 8956E4 MOV [BP-1C],DX
011B 8946E6 MOV [BP-1A],AX
011E C746F60000 MOV [BP-0A],0000

DS:0000 0 1 2 3 4 5 6 7
DS:0000 CD 20 FF 9F 00 EA FF FF
DS:0000 AD DE 1B 05 C5 06 00 00
DS:0010 18 01 10 01 18 01 92 01
DS:0018 01 01 01 00 FF 00 01 00
DS:0020 01 00 01 FF FF FF FF
DS:0028 FF FF FF FF EB 19 E6 11
DS:0030 A2 01 14 00 10 00 F5 19
DS:0038 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00
DS:0048 00 00 00 00 00 00 00 00

2nd iteration: now as cx does not store 0 the jump has not occurred and execution went to next line as shown below:



DOSBox 0.74-3,...

AX:0002 SI:0000 CS:19F5 IP:010F Stack:+0 0000 Flags:7295
BX:0001 DI:0000 DS:19F5
CX:FFFF BP:0000 ES:19F5 HS:19F5
DX:0000 SP:FFFF SS:19F5 FS:19F5

0100 2909 SUB CX,BX
010D E3FA JCXZ 0109
010F B0004C MOV AX,4C00
0112 CD21 INT ZI
0114 31D2 XDR DX,BX
0116 31C9 XDR AX,AX
011B 8956E4 MOV [BP-1C],DX
011B 8946E6 MOV [BP-1A],AX
011E C746F60000 MOV [BP-0A],0000
0123 B846F6 MOV AX,[BP-0A]

DS:0000 0 1 2 3 4 5 6 7
DS:0000 CD 20 FF 9F 00 EA FF FF
DS:0000 AD DE 1B 05 C5 06 00 00
DS:0010 18 01 10 01 18 01 92 01
DS:0018 01 01 01 00 FF 00 01 00
DS:0020 01 00 01 FF FF FF FF
DS:0028 FF FF FF FF EB 19 E6 11
DS:0030 A2 01 14 00 10 00 F5 19
DS:0038 FF FF FF FF 00 00 00 00
DS:0040 05 00 00 00 00 00 00 00
DS:0048 00 00 00 00 00 00 00 00

TASK #3: UNCONDITIONAL JUMPS:

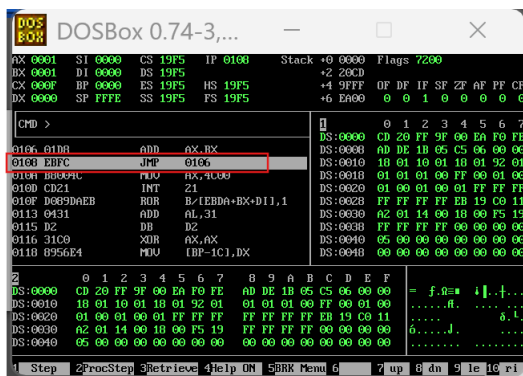
```
1      org 0x0100
2
3      start:
4          mov ax, 0
5          mov bx, 1
6
7      loop_label:
8          add ax, bx
9          jmp loop_label
10
11         mov ax, 0x4c00
12         int 0x21
13
```

The program will run infinitely as there is no stopping condition.

The arrow shows that line 7-9 will run until the program is

Manually stopped.

Demonstrated below:



The screenshot shows the DOSBox 0.74-3 interface. The assembly window displays the following code:

```
0106 0106 ADD AX,BX
0108 EBFC JMP 0106
010A D839AC MOV AX,039A
010B CD21 INT 21
010F D89DAEB ROR B,(EBDA+BX-DI),1
0113 0431 ADD AL,31
0115 D2 DB D2
0116 31C9 XOR AX,AX
011B 895E4 MOV [BP-1C],BX
```

The memory dump shows the state of memory segments DS, ES, FS, and SS. The DS segment is at 0000, ES at 0000, FS at 0000, and SS at 0000. The memory dump also shows the state of the stack and flags.

After this step program will jump and add bx to ax

Infinitely .